GRAPH-BASED INTELLIGENCE FOR ANTI-MONEY LAUNDERING DETECTION

A PROJECT REPORT

Submitted by

VAISHAAL RAJHA.T.C.G (2116231901502)

KAVIN SAINATH (2116231901018)

in partial fulfillment for the award of the degree

of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING (CYBER SECURITY)





RAJALAKSHMI ENGINEERING COLLEGE

ANNA UNIVERSITY: CHENNAI 600 025

OCTOBER 2026

RAJALAKSHMI ENGINEERING COLLEGE

CHENNAI

BONAFIDE CERTIFICATE

Certified that this Project report titled **"GRAPH-BASED INTELLIGENCE FOR ANTI-MONEY LAUNDERING DETECTION"** is the bonafide work of **"VAISHAAL RAJHA.T.C.G (2116231901502), KAVIN SAINATH (2116231901018)"** who carried out the work under my supervision. Certified further that to the best of my knowledge the work reported herein does not form part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE

Dr. P. Tamilselvi, M.E, Ph.D.,

HEAD OF THE DEPARTMENT

Professor

Department of Computer Science and Engineering (Cyber Security)

Rajalakshmi Engineering College
Chennai – 602 105

SIGNATURE

Ms. Rohini C

SUPERVISOR

Assistant Professor

Department of Computer Science and Engineering (Cyber Security)

Rajalakshmi Engineering College
Chennai – 602 105

Submitted to Project Viva-Voce Examination held on 06/10/2026

Internal Examiner

External Examiner

ABSTRACT

Money laundering is a major financial-security problem involving complex transaction patterns, multiple accounts and movement of funds through interconnected networks. Traditional Anti-Money Laundering (AML) systems often analyze transactions independently using predefined rules and thresholds, which may fail to identify relationships between suspicious accounts.

This project proposes a Graph-Based Anti-Money Laundering Detection System using a Graph Convolutional Network (GCN). Financial accounts are represented as graph nodes, while transactions between accounts are represented as directed edges. Transaction behaviour is transformed into account-level features, and the GCN learns information from neighbouring accounts to identify potentially suspicious accounts.

The system is implemented using Python, PyTorch/PyTorch Geometric, NetworkX, Pandas and Streamlit. The Streamlit dashboard provides dataset exploration, model training and evaluation, transaction analysis, network visualization, risk analysis and report/export functionality.

The current implementation processes 200,000 transactions, including 147 fraudulent transaction records. The trained model achieved an accuracy of 99.94%, precision of 25.00%, recall of 3.03%, F1-score of 5.41%, and ROC-AUC of 71.95%. Due to the highly imbalanced synthetic dataset, accuracy alone is not considered sufficient for evaluating AML effectiveness.

The proposed system demonstrates how graph-based machine learning can support suspicious-account identification and investigation by combining transaction behaviour with relationships between accounts.

ACKNOWLEDGEMENT

First, we thank the almighty God for the successful completion of the project. Our sincere thanks to our Chairman **Mr. S. Meganathan B.E., F.I.E.,** for his sincere endeavor in educating us in his premier institution. We would like to express our deep gratitude to our beloved Chairperson **Dr. Thangam Meganathan Ph.D.,** for her enthusiastic motivation which inspired us a lot in completing this project and Vice Chairman **Mr. Abhay Shankar Meganathan B.E., M.S.,** for providing us with the requisite infrastructure.

We also express our sincere gratitude to our college Principal **Dr. S.N. Murugesan M.E., Ph.D.,** and **Dr. P. Tamilselvi M.E., Ph.D., Professor and Head, Department of Computer Science and Engineering (Cyber Security)** and our project guide **Ms. Rohini C, Assistant Professor,** for her encouragement and guiding us throughout the project. We would like to thank our project coordinator **Mr. Benedict J.N., M.E., Assistant Professor (SG),** for his encouragement and guidance towards successful completion of this project and to our parents, friends, all faculty members and supporting staffs for their direct and indirect involvement in successful completion of the project for their encouragement and support.

VAISHAAL RAJHA.T.C.G (2116231901502)

KAVIN SAINATH (2116231901018)

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	**iii**
	LIST OF TABLES	**vii**
	LIST OF FIGURES	**ix**
1	**INTRODUCTION**	**1**
	1.1 BACKGROUND	1
	1.2 PROBLEM STATEMENT	2
	1.3 OBJECTIVES	3
	1.4 SCOPE	3
	1.5 SECURITY REQUIREMENTS / SECURITY CONSIDERATIONS	4
	1.6 SDG ALIGNMENT	4
	1.7 METHODOLOGY OVERVIEW	5
	1.8 ORGANIZATION OF THE REPORT	6
2	**LITERATURE REVIEW**	**7**
	2.1 EXISTING SYSTEM WITH SECURITY APPROACHES	7
	2.2 RESEARCH GAPS	13
	2.3 PROPOSED SYSTEM	13
	2.3.1 OVERVIEW OF THE PROPOSED SYSTEM	13
	2.3.2 OBJECTIVES OF THE PROPOSED SYSTEM	13
	2.3.3 ADVANTAGES OF PROPOSED SYSTEM	14

CHAPTER NO.	TITLE	PAGE NO.
3	**SYSTEM DESIGN**	**15**
	3.1 SYSTEM ARCHITECTURE	15
	3.2 MODULE DESIGN	17
	3.3 DATA FLOW/UML DIAGRAMS	21
	3.4 THREAT MODEL AND ATTACK SURFACE	24
	3.5 SECURITY CONTROLS	26
4	**SYSTEM DESCRIPTION**	**27**
	4.1 DATA COLLECTION	27
	4.2 DATA PREPROCESSING	28
	4.2.1 ALGORITHMS / METHODOLOGY	29
	4.2.2 RESULTS AND DISCUSSION	32
	4.3 IMPLEMENTATION DETAILS	36
	4.4 TESTING AND VERIFICATION	37
5	**CONCLUSION**	**39**
	5.1 FUTURE ENHANCEMENTS	39
	5.2 LIMITATIONS	40
	5.3 SUMMARY OF CONTRIBUTIONS	40
	5.4 SOCIETAL AND ETHICAL CONSIDERATIONS	41
	5.5 RECOMMENDATIONS FOR FURTHER WORK	41
	REFERENCES	**42**
	APPENDIX	**43**

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
1.1	LAUNDERING STAGES AND THEIR GRAPH VIEW	2
1.2	SCOPE OF THE PRESENT WORK	4
2.1	TRADITIONAL AND GRAPH-BASED DETECTION	8
2.2	COMPARISON OF RELATED WORK	9
2.3	TECHNIQUES FOR CLASS IMBALANCE	11
2.4	LESSONS FROM THE LITERATURE AND DESIGN IMPLICATIONS	12
2.5	RESEARCH GAPS AND PROJECT RESPONSE	13
3.1	LAYERS OF THE SYSTEM	16
3.2	MAPPING OF OBJECTIVES TO PROJECT COMPONENTS	19
3.3	FUNCTIONAL REQUIREMENTS	19
3.4	NON-FUNCTIONAL REQUIREMENTS	19
3.5	DESCRIPTION OF MAIN USE CASES	22
3.6	INTERACTION BETWEEN DASHBOARD MODULES AND ARTIFACTS	24
3.7	THREAT MODEL	24
3.8	SECURITY CONCERNS BY COMPONENT	25
3.9	MAPPING OF SECURITY CONTROLS TO THREATS	26
4.1	SUMMARY OF THE DATASET USED	27
4.2	GCN CONFIGURATION	29
4.3	RISK CATEGORIES	31
4.4	PERFORMANCE EVALUATION METRICS	32
4.5	INTERPRETATION OF EVALUATION METRICS	33
4.6	CONFUSION MATRIX TERMS	34
4.7	QUALITATIVE EFFECT OF THE DECISION THRESHOLD	35
4.8	QUALITATIVE COMPARISON OF THE TWO VIEWS	36
4.9	PROJECT FILES AND RESPONSIBILITIES	36

TABLE NO.	TITLE	PAGE NO.
4.10	VERIFICATION CHECKS BY MODULE	37
5.1	FUTURE ENHANCEMENTS AND EXPECTED BENEFIT	40
A.1	GLOSSARY OF TERMS	43
A.2	TECHNOLOGIES USED	44
A.3	LIST OF ABBREVIATIONS	46
A.4	DEPLOYMENT CHECKLIST	46
A.5	DEFINITIONS OF THE EVALUATION METRICS	47
A.6	DASHBOARD PAGE ELEMENTS	48
A.7	SUMMARY OF PREPROCESSING STEPS	48
A.8	REQUIREMENTS AND VERIFICATION CHECKS	49
A.9	STRUCTURAL PATTERNS IN TRANSACTION GRAPHS	49
A.10	COMPARISON OF GRAPH NEURAL NETWORK ARCHITECTURES	50
A.11	KEY FIGURES OF THE PROJECT	51
A.12	OBJECTIVES AND REPORT SECTIONS	51

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
1.1	STAGES OF THE METHODOLOGY	5
2.1	TWO-HOP MESSAGE PASSING TOWARDS A TARGET ACCOUNT	10
3.1	SYSTEM ARCHITECTURE	15
3.2	NAVIGATION BETWEEN DASHBOARD MODULES	20
3.3	DATA FLOW DIAGRAM	21
3.4	USE CASE DIAGRAM	22
3.5	SEQUENCE DIAGRAM FOR ACCOUNT INVESTIGATION	23
3.6	ACTIVITY FLOW OF MODEL TRAINING	24
4.1	NEIGHBOURHOOD OF AN ACCOUNT IN THE TRANSACTION GRAPH (ILLUSTRATIVE)	29
4.2	STRUCTURE OF THE TWO-LAYER GCN	30
4.3	FROM MODEL OUTPUT TO RISK CLASS	31
4.4	EVALUATION METRICS OF THE TRAINED MODEL	32
4.5	CONFUSION MATRIX TERMS	33

CHAPTER 1
INTRODUCTION

1.1 BACKGROUND

Anti-Money Laundering systems are designed to identify suspicious financial activities and support financial investigations.

Traditional AML systems generally analyze individual transactions using:

- Rule-based detection
- Transaction thresholds
- Statistical analysis
- Machine-learning classification
- Suspicious transaction patterns

However, financial transactions are naturally connected. A single account may appear normal when analyzed independently but may become suspicious when its relationships with other accounts are considered. Graph Machine Learning provides an effective approach for representing these relationships.

Money laundering is commonly described as a process with three broad stages. In placement, illicit funds are introduced into the financial system. In layering, the funds are moved through a series of transfers between accounts so that their origin becomes difficult to trace. In integration, the funds re-enter the economy as apparently legitimate wealth. The layering stage is especially relevant to this project, because it produces chains and clusters of connected transactions that are hard to recognise when each transaction is inspected on its own.

A transaction network is a natural graph. Each account can be viewed as a node and each transfer of funds as a directed edge from the sending account to the receiving account. Once the data is represented in this form, a Graph Neural Network can combine the behaviour of an account with information from the accounts it transacts with. This is the central idea explored in this project.

Table 1.1 relates the three stages of money laundering to the way each stage may appear when the transactions are viewed as a graph.

Table 1.1 Laundering Stages and Their Graph View

Stage	Description	Possible appearance in a transaction graph
Placement	Illicit funds enter the financial system.	Accounts that receive unusual inflows from a small number of sources.
Layering	Funds are moved between accounts to hide their origin.	Chains, clusters or loops of connected accounts with repeated transfers.
Integration	Funds re-enter the economy as apparently legitimate money.	Accounts that collect funds from many connected accounts and move them onward.

It should be noted that the patterns in Table 1.1 are descriptive. They illustrate why relationships matter; they are not rules that the proposed system applies directly. Instead, the GCN learns from the data how the behaviour of an account and the behaviour of its neighbours relate to suspicious labels.

In the proposed system:

- Accounts →Graph Nodes
- Transactions →Directed Edges
- Transaction behaviour →Node Features
- Fraudulent/suspicious behaviour →Node Labels
- GCN →Risk Prediction

1.2 PROBLEM STATEMENT

Existing AML systems face the following challenges:

- Transactions are often analyzed independently.
- Relationships between accounts may be ignored.
- Multi-hop transaction chains are difficult to detect.
- Financial fraud datasets are highly imbalanced.
- Accuracy can give a misleading impression of model performance.
- Investigators require visual information about transaction networks.
- Large datasets can increase processing time.

Class imbalance deserves particular attention. When suspicious records form a very small fraction of the data, a model can obtain a very high accuracy simply by labelling almost every account as normal, while still missing most of the suspicious accounts. Evaluation must therefore rely on measures that focus on the minority class.

Investigators face a further practical difficulty. A list of scores or alerts, however accurate, is of limited use unless the investigator can see why an account was flagged. Seeing the transactions of the account, the accounts it is connected to and its position in the surrounding network makes it far easier to judge whether further action is justified. Many research prototypes stop at the prediction and leave this investigation step to separate tools.

Processing time is also a concern. Transaction datasets contain very large numbers of records, and repeating preprocessing or retraining whenever a user opens a page would make an interactive tool unusable. A practical system must therefore separate the expensive steps, such as preprocessing and training, from the lightweight steps of browsing and analysis.

Therefore, a system is required that can analyze both transaction behaviour and relationships between accounts.

1.3 OBJECTIVES

The objectives of the project are to construct a graph from financial transaction data, representing accounts as graph nodes and transactions as directed graph edges; to generate account-level behavioural features; to implement a Graph Convolutional Network that identifies potentially suspicious accounts; to generate account-level risk scores and classify accounts into LOW, MEDIUM and HIGH risk; to provide transaction-level investigation; to visualize account relationships; and to provide report and export functionality.

These objectives fall into three groups. The first group (graph construction, nodes, edges and features) concerns how the transaction data is represented. The second group (the GCN, suspicious-account identification, risk scores and risk classes) concerns how the model produces its output. The third group (transaction-level investigation, visualization, reporting and export) concerns how the output is made useful to an investigator. Section 3.2 shows how each objective is realized in the implementation.

1.4 SCOPE

The project covers dataset loading, data preprocessing, transaction encoding, graph construction, feature generation, GCN model training, model evaluation, risk-score generation, transaction analysis, network visualization, risk analysis, report generation and CSV export.

The current implementation uses the PaySim synthetic dataset and is intended for academic research and demonstration. The system performs account-level (node) classification with a two-layer GCN. Real-time processing, authentication, role-based access and explainability are outside the present scope and are listed as future enhancements in Chapter 5.

Table 1.2 Scope of the Present Work

Within scope	Outside scope (future work)
PaySim synthetic dataset	Realistic privacy-preserving datasets
Account-level node classification	Temporal transaction features
Two-layer Graph Convolutional Network	GraphSAGE and Graph Attention Networks
Streamlit investigation dashboard	Real-time transaction processing
Risk scores and LOW / MEDIUM / HIGH classes	Authentication and role-based access
CSV export and investigation summary	Explainable AI and audit logging

1.5 SECURITY REQUIREMENTS / SECURITY CONSIDERATIONS

The security requirements of the system include input validation, controlled CSV ingestion, separation of data and model files, local storage of model artifacts, protection of financial information, use of synthetic data during development, controlled access to investigation results and secure model storage.

Real-world deployment would require authentication, authorization, encryption, audit logging and secure data retention.

Because a financial-crime detection system handles sensitive information and influences investigative decisions, both the data and the trained model are treated as assets that must be protected. The data must be protected from disclosure and from tampering; the model must be protected from modification, since an altered model could silently change which accounts are flagged. The threats considered and the corresponding controls are discussed in Sections 3.4 and 3.5.

Security is also considered from the point of view of the users of the results. A risk score is an indicator that supports human judgement and not a finding of wrongdoing. The dashboard therefore presents scores together with the underlying transaction evidence, and the report describes risk scores only as investigation indicators.

1.6 SDG ALIGNMENT

S. No.	SDG No. & Goal	Relevant SDG Target	Project Contribution / Alignment
1	SDG 9 – Industry, Innovation and Infrastructure	Target 9.5	Applies graph AI to financial-security monitoring.
2	SDG 16 – Peace, Justice and Strong Institutions	Target 16.4	Supports suspicious-activity investigation and risk prioritization.

SDG 9 encourages innovation and resilient digital infrastructure. By applying graph-based machine learning to financial-security monitoring, the project demonstrates a technique that can strengthen the analytical infrastructure used by financial institutions. SDG 16 calls for strong institutions and for efforts against illicit financial flows. By helping investigators prioritize accounts and understand transaction networks, the project supports the work of institutions that investigate suspicious activity.

1.7 METHODOLOGY OVERVIEW

The work was carried out as a sequence of stages, shown in Fig 1.1. Each stage produces an output that is consumed by the next stage, and the later stages are exposed through the dashboard.

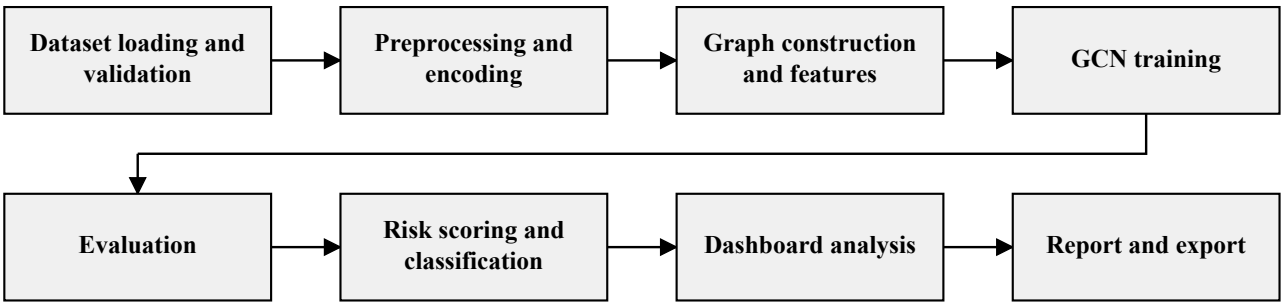


Fig 1.1 Stages of the Methodology

In the first stage the PaySim transaction file is loaded and its fields are validated. In the second stage the transaction types are encoded and account identifiers are created. In the third stage the accounts are mapped to node indices, account-level features are calculated and scaled, and the transactions are turned into directed edges. The fourth and fifth stages train the two-layer GCN and evaluate it using accuracy, precision, recall, F1-score and ROC-AUC. In the sixth stage the output of the model is converted to a risk score and a risk class. The last two stages make the results available for investigation and export.

1.8 ORGANIZATION OF THE REPORT

The remainder of this report is organized as follows.

- **Chapter 2** reviews existing AML approaches and related graph-learning research, identifies research gaps and introduces the proposed system.
- **Chapter 3** presents the system architecture, module design, data flow, threat model and security controls.
- **Chapter 4** describes the dataset, preprocessing, graph construction, the GCN model, risk scoring, implementation, evaluation results and testing.
- **Chapter 5** concludes the report, discusses limitations and lists future enhancements.

CHAPTER 2
LITERATURE REVIEW

2.1 EXISTING SYSTEM WITH SECURITY APPROACHES

Conventional AML systems generally combine rule-based monitoring, statistical analysis and machine-learning classifiers. Common approaches can flag unusually large amounts, rapid movement of funds, repeated transfers or predefined suspicious patterns.

However, these approaches can be limited when suspicious behaviour is distributed across several related accounts.

2.1.1 Rule-Based Monitoring

Rule-based monitoring is the traditional foundation of transaction monitoring. Analysts and compliance teams define conditions, for example a limit on the value of a single transfer or a limit on the number of transfers in a period, and a transaction that satisfies a condition generates an alert. Rules are transparent and easy to explain, which is valuable in a regulated environment, and they can be adjusted quickly when a new typology is identified.

The weaknesses of rules are well known. Fixed thresholds can be learned and avoided by people who wish to launder money, for instance by keeping each transfer below a limit. Rules also examine a transaction or an account in isolation, so they cannot easily represent behaviour that is spread across a group of accounts. Finally, rules that are tuned to avoid missing suspicious activity tend to produce many alerts that investigators must review and dismiss.

2.1.2 Statistical and Machine-Learning Approaches

Statistical analysis and machine-learning classifiers improve on fixed rules by learning from historical data. Techniques such as logistic regression, decision trees, random forests and neural networks can combine many features of a transaction or an account into a single score. These models can capture patterns that are difficult to express as simple rules and can be retrained as new data becomes available.

Nevertheless, such models usually treat each record as an independent row in a table. Information about how an account is connected to other accounts has to be converted by hand into extra columns, such as counts of neighbours, before the model can use it. This feature engineering is laborious, and it is difficult to capture multi-hop structure in this way.

2.1.3 Graph-Based Approaches

Graph-based approaches represent the financial network explicitly. Accounts are nodes, transactions are edges, and the learning algorithm works on the network itself. Graph Neural Networks, and Graph Convolutional Networks in particular, learn a representation of every node by combining its own features with those of its neighbours, so relational information is incorporated automatically.

Table 2.1 Traditional and Graph-Based Detection

Aspect	Traditional approach	Graph-based approach
Unit of analysis	Individual transaction or account	Account within its network
Relationships	Ignored or hand-crafted	Learned from connectivity
Multi-hop behaviour	Difficult to represent	Captured by stacked layers
Investigation support	Alert lists	Scores with network context
Main difficulty	Isolated view, many alerts	Class imbalance, scalability

2.1.4 Review of Related Work

A growing body of research applies graph learning to this problem. The works reviewed for this project are summarized below.

Graph Convolutional Networks on Bitcoin data [1]. Weber et al. present results from a binary classification task that predicts illicit transactions using Logistic Regression, Random Forest, Multilayer Perceptrons and Graph Convolutional Networks. They contribute the Elliptic Data Set, a time-series graph of over 200K Bitcoin transactions (nodes), 234K directed payment flows (edges) and 166 node features. Their results show the superiority of Random Forest, but they also invite work that combines the strengths of Random Forest and graph methods. This motivates the use of a GCN in the present project while recognizing that graph methods are not automatically superior.

Class-imbalanced detection [2]. The work on GCN with focal loss, as its title indicates, addresses class-imbalanced AML detection. It is relevant because the dataset used in this project is also highly imbalanced; this project instead uses class weighting with BCEWithLogitsLoss.

Surveys of graph neural networks for fraud [3]. The systematic review of Graph Neural Networks for financial fraud detection provides an overview of how graph models are applied to fraud problems, and supports the choice of a graph-based approach for account-level risk prediction.

Label-efficient learning [4]. Graph contrastive pre-training for AML proposes a framework that mines supervised signals from a label-free transaction network, which significantly reduces the dependence on annotations. This is relevant because labelled suspicious records are scarce in AML data.

Subgraph learning [5]. Bellei et al. argue that AML is inherently a subgraph problem and introduce Elliptic2, a dataset with 122K labeled subgraphs of Bitcoin clusters within a background graph of 49M node clusters and 196M edge transactions. Their work shows that the local structure, or shape, of a transaction neighbourhood carries information about illicit activity. The local-neighbourhood network view of the present dashboard follows the same intuition.

Attention-based models [6] and reference implementations [7], [8]. The global-local graph attention work targets Bitcoin AML detection with attention-based graph models, which relates to the Graph Attention Network extension listed in Section 5.1. The two GitHub projects provide an Elliptic GNN implementation and a heterogeneous-graph study for AML, and served as reference material for graph-based AML experimentation.

Table 2.2 Comparison of Related Work

Ref.	Source	Focus and relevance to this project
[1]	arXiv	GCN versus LR, RF and MLP on the Elliptic Data Set; motivates GCN for relational information.
[2]	IEEE Xplore	GCN with focal loss for class-imbalanced AML; relevant to the imbalance problem.
[3]	ScienceDirect	Systematic review of GNNs for financial fraud detection; supports the graph-based approach.
[4]	Springer	Graph contrastive pre-training for AML; reduces dependence on labels.
[5]	arXiv	Subgraph representation learning with Elliptic2; local transaction shapes.
[6]	Scientific Reports	Global-local graph attention for Bitcoin AML; relates to future GAT work.
[7]	GitHub	Elliptic GNN implementation used as a reference project.
[8]	GitHub	Heterogeneous graph modelling for AML; reference study.

The above works show that relational information is valuable for AML, and also that class imbalance, limited labels and evaluation practice remain open concerns. These observations shape the design and the cautious interpretation of results in this project.

2.1.5 Fundamentals of Graph Neural Networks

A graph consists of a set of nodes and a set of edges that connect pairs of nodes. In a directed graph every edge has a direction, which suits financial transactions because funds flow from a sender to a receiver. Each node can carry a feature vector, and each node may be associated with a label, such as whether the account was involved in fraudulent activity.

Graph Neural Networks learn node representations through message passing. In every layer, each node collects the representations of its neighbours, combines them, and updates its own representation. After one layer a node reflects its direct neighbours; after two layers it reflects neighbours of neighbours, as shown in Fig 2.1. The final representation of a node is then used for a prediction task, which in this project is the classification of an account as suspicious or normal.

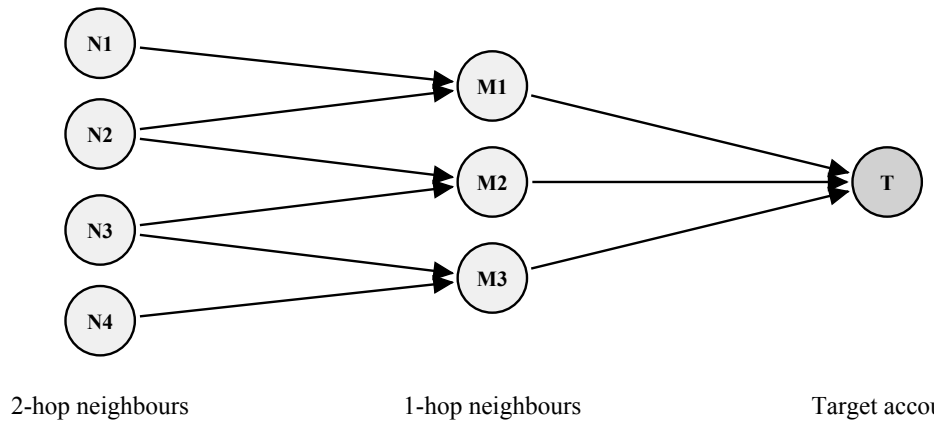


Fig 2.1 Two-Hop Message Passing Towards a Target Account

The Graph Convolutional Network is one of the simplest and most widely used message-passing architectures. At each layer it averages the representations of a node and its neighbours, with a normalization that accounts for the number of connections, and then applies a learnable linear transformation followed by a non-linear function. Stacking two such layers allows information from accounts up to two transactions away to influence the representation of an account. Other architectures, such as GraphSAGE and Graph Attention Networks, change how neighbours are sampled or weighted; they are discussed as future enhancements in Chapter 5.

2.1.6 Handling Class Imbalance

In AML data the suspicious class is extremely rare. Several techniques are used to prevent a model from ignoring the minority class. Class weighting multiplies the loss for rare examples by a larger factor, so that mistakes on suspicious accounts cost more. Resampling

changes the training set by duplicating minority examples or removing majority examples. Specialised loss functions, such as focal loss, reduce the contribution of easy examples. Finally, the decision threshold can be adjusted after training so that more accounts are flagged.

Table 2.3 Techniques for Class Imbalance

Technique	Idea	Use in this project
Class weighting	Larger loss for the rare class	Used
Resampling	Change class proportions in training data	Not used
Focal loss	Down-weight easy examples	Not used (see [2])
Threshold tuning	Change the operating point after training	Future enhancement

2.1.7 Evaluation Metrics for Imbalanced Data

Accuracy counts the fraction of correct predictions and is therefore dominated by the majority class. Precision measures how many of the flagged accounts are truly suspicious, recall measures how many of the suspicious accounts are found, and the F1-score is the harmonic mean of the two. ROC-AUC measures how well the model ranks suspicious accounts above normal accounts across all possible thresholds. For strongly imbalanced problems the precision-recall curve and its area (PR-AUC) are often more informative than ROC-AUC, because they focus entirely on the minority class; PR-AUC is therefore listed as a future enhancement.

2.1.8 Explainability and Visual Analytics

Model output is only useful to an investigator if it can be understood. In AML, a score that cannot be explained is difficult to defend to a supervisor, and an analyst cannot decide what to do with an account unless the reasons for suspicion are visible. Two complementary approaches are therefore used in practice. Explainable AI methods try to identify which features or neighbours most influenced a prediction. Visual analytics presents the data and the network so that a human can judge the evidence directly.

The present project follows the second approach. It does not implement an Explainable AI method, which is listed as future work, but it places the transaction history, the connected accounts and the local network next to every score. This allows the investigator to see, for instance, whether a highly scored account sends and receives funds from many other accounts, or whether it is isolated.

2.1.9 Privacy and Regulatory Considerations

Financial institutions operate under regulations that require them to monitor transactions and report suspicious activity, and at the same time they are responsible for protecting customer data. A detection system must therefore balance two obligations. It should be effective enough to support the monitoring requirement, and it should limit the exposure of personal and financial information. Using synthetic data during development, restricting access to model files and results, and keeping an audit trail in deployment are all measures that serve the second obligation.

It is also important to avoid treating a model output as a decision. Automated scores can contain errors and can reflect biases in the data on which the model was trained. Human review of flagged accounts is therefore an essential part of any responsible AML workflow, and the proposed system is designed to support that review.

2.1.10 Datasets Used in Graph-Based AML Research

Research on graph-based AML relies on a small number of datasets, because real transaction data is confidential. The Elliptic Data Set [1] and Elliptic2 [5] are labelled graphs derived from Bitcoin transactions, and they are among the largest labelled transaction datasets available. Synthetic simulators generate transactions together with labelled fraudulent behaviour; PaySim, used in this project, is of this kind. Each type of dataset has its advantages. Real cryptocurrency graphs contain genuine behaviour but follow the conventions of the blockchain, while synthetic data can be generated for the structure and scale that a study requires but may simplify real behaviour.

2.1.11 Lessons from the Review

Table 2.4 Lessons from the Literature and Design Implications

Lesson from the literature	Implication for this project
Relational information helps but is not automatically superior [1]	Evaluate with several metrics and avoid claiming superiority.
Class imbalance is a central difficulty [2]	Use class weighting and report precision, recall, F1-score and ROC-AUC.
Labels are scarce [4]	Treat labelled suspicious records as a limited resource.
Local structure carries information [5]	Provide a local neighbourhood network view.
Graph models are widely applied to fraud [3]	Adopt a graph-based approach for account-level risk.

2.2 RESEARCH GAPS

- Limited use of relational information between accounts.
- Difficulty representing multi-hop transaction behaviour.
- Severe class imbalance.
- Difficulty reducing false negatives.
- Investigation workflows are often separated from model predictions.
- Explainability and visualization remain important requirements.

Many studies concentrate on model accuracy and do not provide a workflow for the analyst who must act on the predictions. This project addresses that gap by placing the trained model behind an interactive dashboard that links risk scores to transaction evidence and to the surrounding network.

Table 2.5 Research Gaps and Project Response

Research gap	**Response in this project**
Limited use of relational information	Accounts and transactions are modelled as a graph.
Multi-hop behaviour is hard to represent	A two-layer GCN aggregates two-hop neighbourhoods.
Severe class imbalance	Class weighting and metrics beyond accuracy.
Predictions separated from investigation	Dashboard with transaction, network and risk views.
Need for visual evidence	Local neighbourhood network visualization.

2.3 PROPOSED SYSTEM

2.3.1 OVERVIEW OF THE PROPOSED SYSTEM

The proposed system combines transaction preprocessing, graph construction, GCN-based node classification and an interactive Streamlit dashboard.

Transactions from the PaySim dataset are validated, encoded and aggregated into account-level features. A graph is then built in which accounts are nodes and transactions are directed edges. The GCN is trained on this graph, and its output is converted into an account-level risk score and a LOW, MEDIUM or HIGH risk category.

2.3.2 OBJECTIVES OF THE PROPOSED SYSTEM

The central objective is to prioritize potentially suspicious accounts by combining their own transaction behaviour with information propagated from neighbouring accounts.

- Represent the transaction network as a graph of accounts and transfers.

- Learn account representations that reflect neighbouring accounts.
- Provide risk scores that help investigators prioritize accounts.
- Present model metrics together with transaction and network evidence.

2.3.3 ADVANTAGES OF PROPOSED SYSTEM

- Captures relationships between accounts.
- Produces account-level risk scores.
- Provides network visualization.
- Combines model metrics with transaction evidence.
- Provides an interactive investigation dashboard.
- Can be extended with advanced GNN architectures.

The advantages listed above should be read together with the limitations discussed in Chapter 4. The current implementation demonstrates the approach on synthetic data; its measured recall is low, and the dashboard is a research and demonstration tool rather than a production system.

CHAPTER 3
SYSTEM DESIGN

3.1 SYSTEM ARCHITECTURE

The architecture of the proposed system is shown in Fig 3.1. The PaySim dataset is preprocessed and converted into account-level features and a transaction graph, which is passed to the Graph Convolutional Network. The resulting risk predictions are classified and presented through the Streamlit dashboard for transaction, network and risk analysis, and for report and export.

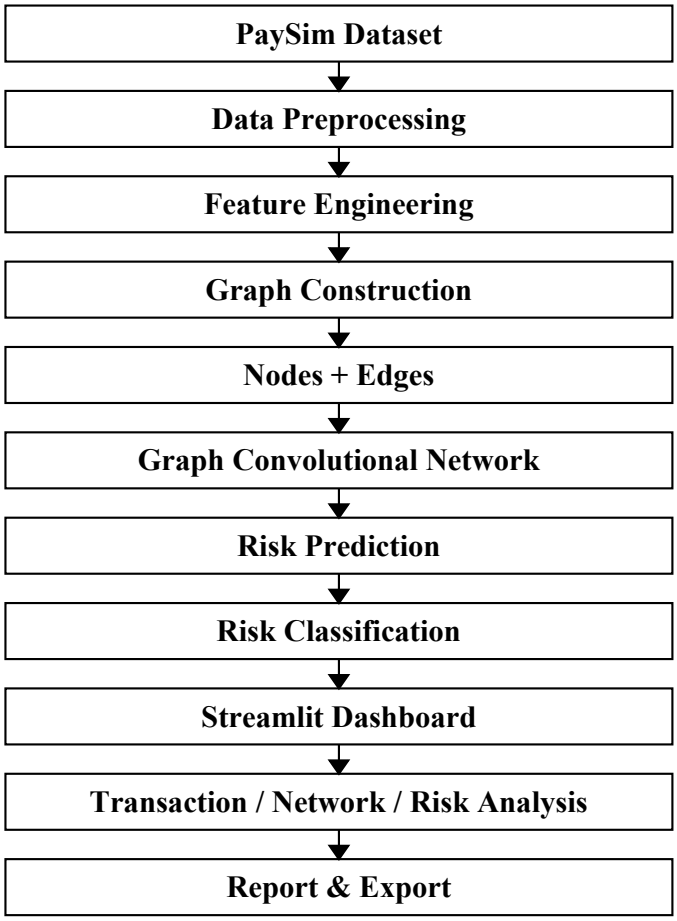


Fig 3.1 System Architecture

The architecture can be understood in four groups of stages.

Data stage. The PaySim transaction records are loaded and validated. Transaction types are encoded and account identifiers are created.

Graph stage. Accounts are mapped to node indices, account-level features are calculated and scaled, and transactions become directed edges. Account-level labels are created from the fraudulent transaction records.

Model stage. The two-layer GCN produces a probability-like risk score for every account, and the score is mapped into LOW, MEDIUM or HIGH.

Presentation stage. The Streamlit dashboard loads the saved model artifacts and presents dataset, transaction, network and risk views, together with report and export options.

3.1.1 Design Principles

Four principles guided the design. First, the expensive steps (preprocessing and training) are separated from the interactive steps, so that navigating the dashboard does not trigger retraining. Second, model artifacts are stored in a dedicated directory, separate from the data, so that each can be protected independently. Third, every score shown to the user is accompanied by evidence, namely the transactions and the neighbouring accounts. Fourth, the system uses synthetic data during development so that no real customer information is exposed.

3.1.2 Layered View

The system can also be viewed as three layers. The data layer holds the transaction file and the saved artifacts. The processing layer contains the preprocessing, graph-building, training and prediction code. The presentation layer is the Streamlit dashboard. The layers communicate only through well-defined files and function calls, which keeps the dashboard independent of the details of the model code and makes it possible to replace the GCN with another model in the future.

Table 3.1 Layers of the System

Layer	Responsibilities	Main files / artifacts
Data	Store transactions and saved artifacts	data/, models/, outputs/
Processing	Preprocessing, graph construction, training, prediction	preprocessing.py, graph_builder.py, model.py, train.py, predict.py
Presentation	Interactive pages, charts, network view, export	app.py

3.1.3 Design Assumptions and Constraints

The design rests on several assumptions. The transaction data is assumed to contain, for each transaction, the sending and receiving accounts, the transaction type, the amount and a fraud indicator, as in the PaySim dataset. The system assumes that labels derived from the fraud indicator are the target for supervised learning. Constraints include the size of the dataset, which limits what can be displayed interactively, and the need to keep the dashboard responsive on an ordinary computer. The system is a prototype for academic use, so

authentication, encryption and audit logging, although recommended, are outside the implemented scope.

3.2 MODULE DESIGN

HOME: Shows project overview, dataset status, model status, key metrics and high-level architecture.

DATASET: Shows dataset information, transaction records, transaction types, fraud distribution, statistics and sample records.

MODEL TRAINING & EVALUATION: Provides dataset configuration, training epochs, learning rate, model training and evaluation metrics.

TRANSACTION ANALYSIS: Provides account search, risk score, transaction count, connected accounts, financial summary and recent transactions.

NETWORK VISUALIZATION: Displays account nodes, transaction edges, connected accounts and local transaction neighbourhood.

RISK ANALYSIS: Displays LOW, MEDIUM and HIGH risk accounts, risk distribution and top-risk accounts.

REPORT / EXPORT: Provides suspicious-account export, prediction export and investigation summary.

The following subsections describe each module in more detail, covering its purpose, what the user does in it and the design considerations behind it.

3.2.1 Home Module

The Home module is the entry point of the dashboard. Its purpose is to orient the user: it summarizes what the project does, shows whether a dataset and a trained model are available, and displays the key evaluation metrics so that the quality of the current model is visible immediately. A high-level view of the architecture is also shown. Because the module reads only saved information, it opens quickly and does not trigger any heavy computation.

3.2.2 Dataset Module

The Dataset module helps the user understand the data before using the model. It presents dataset information, the transaction records, the distribution of transaction types and the distribution of fraudulent and non-fraudulent records, together with summary statistics and sample records. Viewing the fraud distribution makes the class imbalance visible at once, which is important for interpreting the evaluation metrics later. Caching is used so that the dataset is not repeatedly reprocessed when the user returns to this page.

3.2.3 Model Training and Evaluation Module

This module lets the user configure and run the training process. The user can set the dataset configuration, the number of training epochs and the learning rate, train the model and view the evaluation metrics: accuracy, precision, recall, F1-score and ROC-AUC. After training, the model parameters, graph data, metrics and scaler are written to the models directory so that other modules can load them without retraining.

3.2.4 Transaction Analysis Module

The Transaction Analysis module supports the investigation of an individual account. The user searches for an account and the module shows its risk score, its transaction count, the accounts it is connected to, a financial summary and its recent transactions. This gives the investigator the evidence needed to interpret the score: whether the account has few or many transactions, with whom it transacts and how much money is involved.

3.2.5 Network Visualization Module

The Network Visualization module shows the account as a node together with its transaction edges and its connected accounts, forming the local transaction neighbourhood. The displayed neighbourhood is deliberately limited in size. A full transaction graph with hundreds of thousands of edges cannot be drawn in an interactive view in a readable way, whereas a local neighbourhood is small enough to be understood and keeps the page responsive.

3.2.6 Risk Analysis Module

The Risk Analysis module provides a portfolio view of the model output. It displays the accounts in the LOW, MEDIUM and HIGH risk categories, the distribution of risk scores and the list of top-risk accounts. An investigator with limited time can start from the top-risk accounts and then move to the Transaction Analysis and Network Visualization modules for detail.

3.2.7 Report and Export Module

The Report / Export module converts the results into files that can be shared or processed further. It provides the export of suspicious accounts, the export of predictions and an investigation summary. Exports are written in CSV format, which is easy to open in common tools, and are stored in the outputs directory.

3.2.8 Objective Mapping and Requirements

Table 3.2 maps each project objective to the component of the implementation that realizes it.

Table 3.2 Mapping of Objectives to Project Components

Objective	Realized by
Construct graph, nodes and edges	preprocessing.py, graph_builder.py
Generate account-level features	preprocessing.py
Implement the GCN and train it	model.py, train.py
Generate risk scores and risk classes	predict.py, Risk Analysis module
Transaction-level investigation	Transaction Analysis module
Visualize account relationships	Network Visualization module
Report and export functionality	Report / Export module

The functional requirements derived from the objectives are listed in Table 3.3, and the non-functional requirements in Table 3.4.

Table 3.3 Functional Requirements

ID	Requirement
FR1	The system shall load and validate the transaction dataset.
FR2	The system shall encode transaction types and create account identifiers.
FR3	The system shall build a directed graph with accounts as nodes and transactions as edges.
FR4	The system shall train and evaluate a two-layer GCN on the graph.
FR5	The system shall produce a risk score between 0 and 1 for each account.
FR6	The system shall classify accounts as LOW, MEDIUM or HIGH risk.
FR7	The system shall allow an account to be searched and its transactions reviewed.
FR8	The system shall visualize the local transaction neighbourhood of an account.
FR9	The system shall export suspicious accounts and predictions in CSV format.

Table 3.4 Non-Functional Requirements

Attribute	Requirement
Performance	Navigation between pages shall not retrain the model; preprocessing results shall be cached.
Usability	Results shall be presented through an interactive dashboard with clearly separated pages.

Attribute	Requirement
Security	Inputs shall be validated; model files shall be stored separately from data.
Maintainability	Preprocessing, graph building, model, training and prediction shall be separate modules.
Reproducibility	Model parameters, graph data, metrics and scaler shall be saved for reuse.
Interpretability	Scores shall be accompanied by transaction and network evidence.

3.2.9 Software and Hardware Environment

The system is implemented in Python using PyTorch and PyTorch Geometric for the GCN, NetworkX for graph handling, Pandas for data processing and Streamlit for the dashboard. The project runs on a standard desktop or laptop computer; a graphics processor is not required for the dashboard, although it can reduce training time for larger graphs.

3.2.10 User Interface Design

The interface is organized as a set of pages reached from a navigation menu, with one page per module. Each page follows the same pattern: controls at the top (for example the account search box or the training settings), followed by summary figures and then detailed tables or charts. Presenting summary information first allows a user to decide quickly whether more detail is needed. Colour and labels are used consistently, in particular for the LOW, MEDIUM and HIGH categories, so that the meaning of a category is recognised on every page.

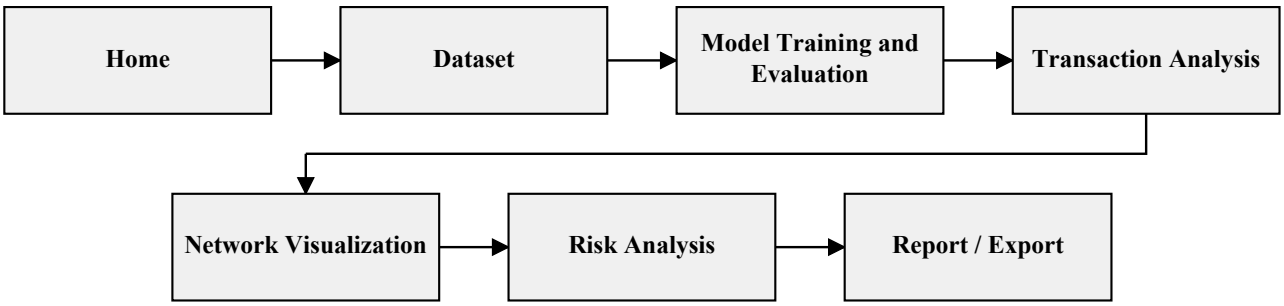


Fig 3.2 Navigation Between Dashboard Modules

A typical path through the dashboard is shown in Fig 3.2. A user first checks the status of the dataset and the model, trains the model if necessary, looks at the overall risk distribution, investigates individual accounts, and finally exports the results. The modules can, however, be opened in any order, because each one reads only saved artifacts.

3.3 DATA FLOW/UML DIAGRAMS

The flow of data from the user interaction to the final report and export is shown in Fig 3.3.

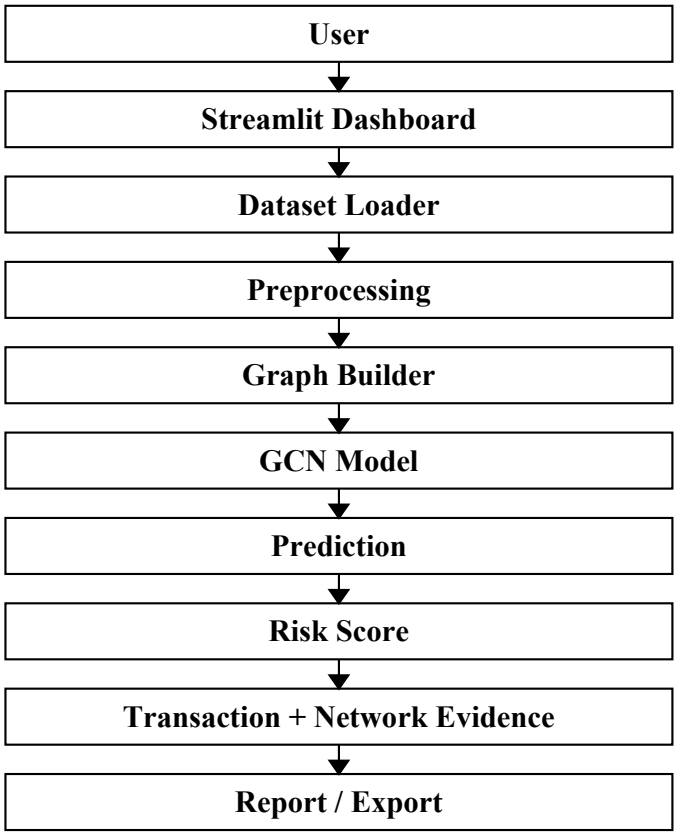


Fig 3.3 Data Flow Diagram

The user interacts only with the dashboard. The dashboard calls the dataset loader and preprocessing steps, the graph builder and the trained GCN model, and finally presents the risk score together with the transaction and network evidence that supports it.

3.3.1 Use Case Diagram

Fig 3.4 shows the interactions between the investigator, who is the main user of the system, and the dashboard. Each use case corresponds to one of the dashboard modules.

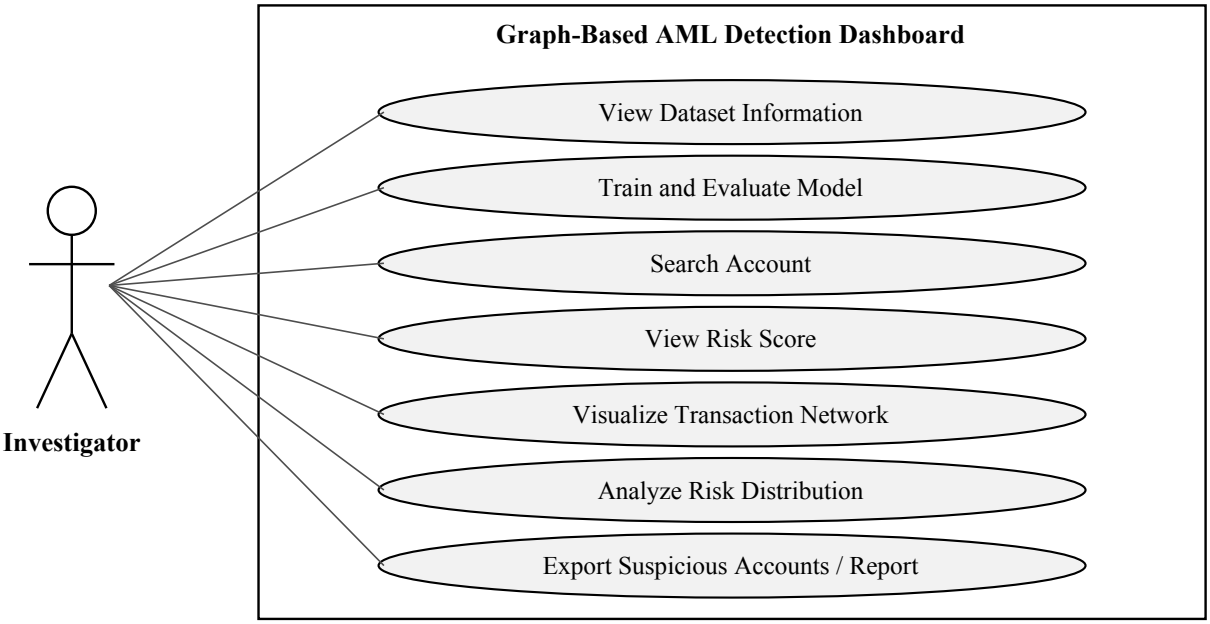


Fig 3.4 Use Case Diagram

Table 3.5 Description of Main Use Cases

Use case	Description
Train and evaluate model	The investigator configures epochs and learning rate, trains the GCN and reviews the evaluation metrics.
Search account	The investigator enters an account and the system retrieves its risk score and transaction summary.
Visualize network	The system displays the local neighbourhood of the selected account.
Analyze risk distribution	The system shows LOW, MEDIUM and HIGH risk accounts and the top-risk accounts.
Export results	The system writes suspicious accounts and predictions to CSV files.

3.3.2 Sequence of an Account Investigation

Fig 3.5 shows the sequence of interactions when an investigator examines an account. The dashboard retrieves the processed graph and features from the data pipeline, requests the risk score from the trained model, and returns the score, the risk class and the supporting evidence to the user.

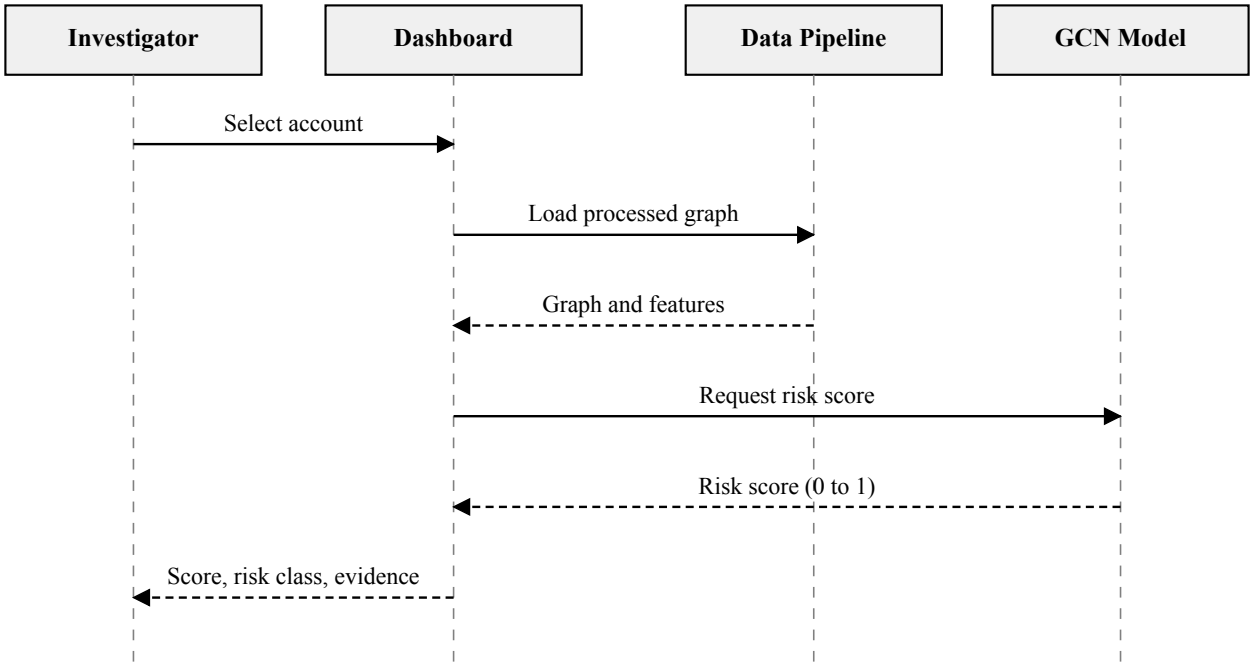


Fig 3.5 Sequence Diagram for Account Investigation

3.3.3 Data Storage Design

Data is organized in three directories. The data directory holds the input transaction file. The models directory holds the trained GCN parameters (gcn_model.pt), the saved graph data (graph_data.pt), the evaluation metrics and training information (metrics.json) and the feature scaler (scaler.pkl). The outputs directory holds the exported CSV files. Keeping these locations separate allows different access controls to be applied to inputs, model files and results.

3.3.4 Training Activity Flow

The activity flow of model training is shown in Fig 3.6. It follows the stages of the methodology, with the difference that the artifacts are saved at the end so that all other modules can use them.

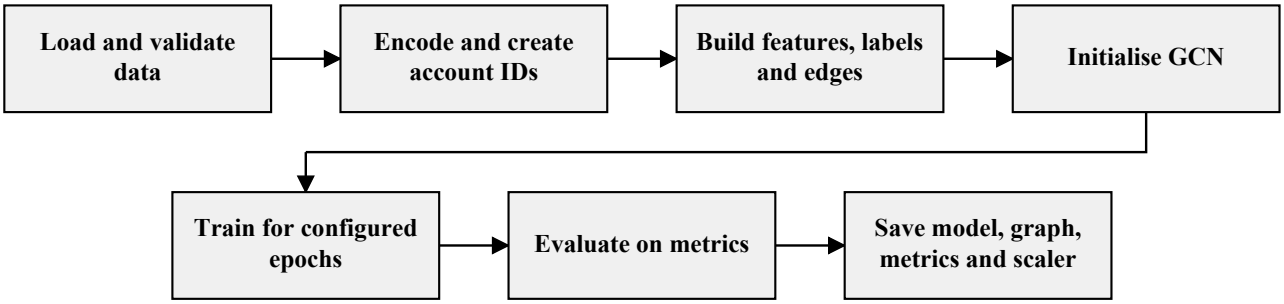


Fig 3.6 Activity Flow of Model Training

3.3.5 Module Interaction Summary

Table 3.6 Interaction Between Dashboard Modules and Artifacts

Module	Reads	Writes
Home	metrics.json	None
Dataset	Transaction data (cached)	None
Model Training and Evaluation	Transaction data	All model artifacts
Transaction Analysis	Graph data, model, scaler	None
Network Visualization	Graph data	None
Risk Analysis	Model, graph data, scaler	None
Report / Export	Predictions	CSV files in outputs

Only the training module writes to the models directory. All other modules read from it. This simple rule makes it easy to protect the model files, because write access is required by one component only.

3.4 THREAT MODEL AND ATTACK SURFACE

The threats considered for the system, their risk levels and the corresponding mitigations are summarized in Table 3.7.

Table 3.7 Threat Model

Threat	Risk	Mitigation
Malicious CSV	High	Input validation
Model tampering	High	Restrict model access
Data leakage	High	Encryption/access control
Dashboard abuse	Medium	Authentication
Adversarial transactions	High	Model monitoring/retraining

The attack surface of the system consists of the dataset upload path, the stored model artifacts, the dashboard itself and the data produced by the system.

- **Malicious CSV.** A crafted input file could contain malformed or unexpected fields. Validating transaction fields before processing reduces this risk.
- **Model tampering.** Altered model files could change predictions. Keeping model files in a separate directory with restricted access reduces this risk.
- **Data leakage.** Transaction data and exported reports contain financial information. Encryption and access control are required for real deployments.
- **Dashboard abuse.** Unauthorized use of the dashboard could expose investigation results. Authentication is the recommended control.
- **Adversarial transactions.** Launderers may adapt their behaviour to avoid detection. Monitoring the model and retraining it periodically addresses this threat.

3.4.1 Component-Wise Analysis

Table 3.8 considers the components of the system and the main concern for each.

Table 3.8 Security Concerns by Component

Component	Main concern	Recommended control
Dataset loader	Malformed or malicious input	Input validation, controlled CSV ingestion
Preprocessing	Unexpected values corrupting features	Field checks and type validation
Model files	Tampering or replacement	Restricted access, separate directory
Dashboard	Unauthorized access to results	Authentication, role-based access
Exported files	Disclosure of financial information	Access control, encryption, retention rules

3.4.2 Risk Assessment

Three of the five threats are rated High because their consequences affect the integrity of the whole system. Malicious input can corrupt the processing pipeline, a tampered model can silently change every prediction, and leaked data exposes financial information. Dashboard abuse is rated Medium because, in a prototype that uses synthetic data, the immediate harm is smaller, but the rating would rise in a real deployment. Adversarial transactions are rated High because they directly aim to defeat the detection capability, which is the purpose of the system.

3.5 SECURITY CONTROLS

- Controlled data directories
- Synthetic financial data
- Input validation
- Separate model/data directories
- Risk scores used only as investigation indicators
- Secure deployment recommendations

Risk scores are used only as indicators to prioritize accounts for investigation. They are not evidence of wrongdoing, and a human investigator reviews the transaction and network evidence before any conclusion is drawn. The use of synthetic data during development avoids exposure of real customer information.

3.5.1 Mapping of Controls to Threats

Table 3.9 Mapping of Security Controls to Threats

Control	**Threats addressed**
Input validation	Malicious CSV, corrupted features
Separate model and data directories	Model tampering, unintended data exposure
Controlled data directories	Data leakage
Synthetic data during development	Data leakage
Scores used only as indicators	Over-reliance on automated output
Monitoring and retraining	Adversarial transactions, model drift

3.5.2 Secure Deployment Recommendations

The current system is a prototype. Before any real-world use, the following measures would be required: authentication and role-based authorization for the dashboard; encryption of data at rest and in transit; audit logging of who viewed or exported which results; secure retention and deletion of financial data; integrity checks on model files; and periodic evaluation of the model against new behaviour. These measures are consistent with the future enhancements listed in Chapter 5.

CHAPTER 4
SYSTEM DESCRIPTION

4.1 DATA COLLECTION

The project uses the PaySim synthetic financial transaction dataset.

Current implementation:

- Transactions processed: 200,000
- Fraudulent transactions: 147
- Fraud rate: approximately 0.07%
- Accounts represented as graph nodes
- Transactions represented as directed graph edges

The fraud rate of approximately 0.07% shows that the data is highly imbalanced: only 147 of the 200,000 processed transactions are fraudulent. This imbalance strongly influences the design of the loss function and the choice of evaluation metrics.

4.1.1 Why a Synthetic Dataset

Real financial transactions are confidential, and datasets that contain real customer activity are rarely available for academic work. Synthetic datasets simulate transactions and include labelled fraudulent behaviour, which allows models and tools to be developed and demonstrated without exposing real data. The disadvantage is that synthetic data may not reproduce all the complexity of real money-laundering behaviour, so results obtained on it should not be assumed to carry over to real institutions. This limitation is acknowledged throughout the report.

4.1.2 Dataset Summary

Table 4.1 Summary of the Dataset Used

Item	**Value**
Dataset	PaySim (synthetic)
Transactions processed	200,000
Fraudulent transactions	147
Fraud rate	Approximately 0.07%
Graph nodes	Accounts
Graph edges	Transactions (directed)

4.2 DATA PREPROCESSING

The preprocessing process includes:

1. Loading transaction data.
2. Validating transaction fields.
3. Encoding transaction types.
4. Creating account IDs.
5. Mapping accounts to graph-node indices.
6. Calculating account-level features.
7. Constructing transaction edges.
8. Creating account-level labels.
9. Scaling numerical features.

The scaling object used during training is saved as scaler.pkl, so that the same transformation is applied to data at inference time.

4.2.0 Description of the Preprocessing Steps

1. Loading transaction data. The transaction file is read from the data directory into a tabular structure using Pandas. Loading is performed once and the result is cached by the dashboard.

2. Validating transaction fields. Required fields are checked for presence and expected type before further processing. Records that do not satisfy the checks are not allowed to corrupt later stages. This step is the first line of defence against malformed or malicious CSV input.

3. Encoding transaction types. Transaction types are textual categories and are converted into numeric codes so that they can be used by the model.

4. Creating account IDs. Every account that appears as a sender or receiver is given a unique identifier, so that the same account is recognised wherever it occurs.

5. Mapping accounts to graph-node indices. The graph library requires nodes to be numbered consecutively. Each account identifier is therefore mapped to an integer node index.

6. Calculating account-level features. The behaviour of each account across its transactions is summarized into a feature vector, for example the number of transactions and the amounts involved.

7. Constructing transaction edges. Each transaction becomes a directed edge from the node index of the sender to the node index of the receiver.

8. Creating account-level labels. An account is labelled using the fraudulent or suspicious

behaviour recorded in its transactions, producing the target for supervised training.

9. Scaling numerical features. Numerical features are scaled so that features with large ranges do not dominate those with small ranges. The scaler is saved for use at inference time.

4.2.1 ALGORITHMS / METHODOLOGY

A. Graph Construction

Each financial account becomes a node. Each transaction becomes a directed edge. For example, Account A →Account B represents the movement of funds from Account A to Account B.

Formally, the transaction network is a directed graph G = (V, E), where V is the set of accounts and E is the set of transactions. Every node carries a feature vector built from its transaction behaviour and a label that indicates whether suspicious behaviour was observed. Fig 4.1 illustrates the neighbourhood of an account.

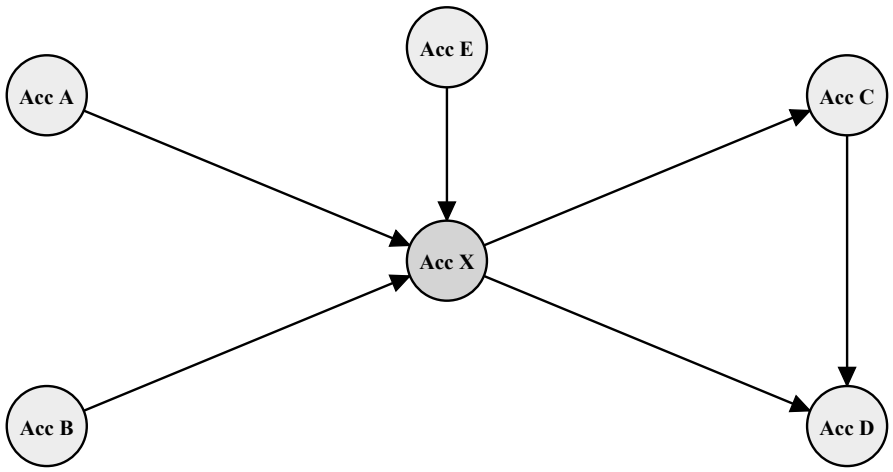


Fig 4.1 Neighbourhood of an Account in the Transaction Graph (Illustrative)

B. Graph Convolutional Network

The project uses a two-layer GCN with the configuration given in Table 4.2.

Table 4.2 GCN Configuration

Parameter	**Value**
First GCN layer	64 hidden dimensions
Second GCN layer	32 hidden dimensions
Dropout	0.3
Loss function	BCEWithLogitsLoss
Imbalance handling	Class weighting

The overall structure of the model is shown in Fig 4.2.

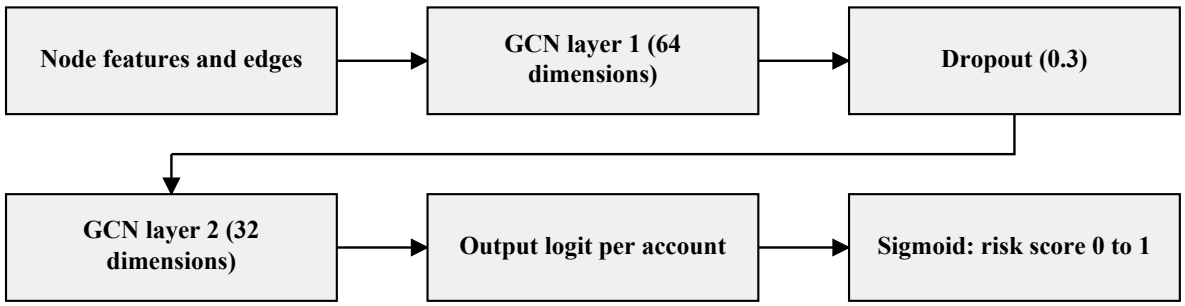


Fig 4.2 Structure of the Two-Layer GCN

The GCN aggregates information from neighbouring accounts to create better representations of each account. In the standard formulation, each layer updates the node representations as:

$$H(l+1) = \sigma(D^{-1/2} \hat{A} D^{-1/2} H(l) W(l))$$

Here Â is the adjacency matrix with self-loops, D is its degree matrix, H(l) holds the node representations at layer l, W(l) is a learnable weight matrix and σ is a non-linear function. With two layers, the representation of an account is influenced by accounts up to two hops away, which allows multi-hop transaction behaviour to be reflected in the prediction.

The self-loops in Â ensure that an account keeps its own information when it is combined with that of its neighbours. The normalization by the degree matrix prevents accounts with many connections from dominating the result simply because they have more neighbours. The weight matrix W(l) is learned during training and determines which combinations of features are useful for the task. Dropout randomly switches off a fraction of the hidden values during training, with a probability of 0.3 in this project, which reduces over-fitting.

Training uses BCEWithLogitsLoss, which combines a sigmoid function with binary cross-entropy. Class weighting increases the contribution of the rare suspicious class so that the model does not simply favour the majority class. For a node with label y and predicted probability p, the weighted loss can be written as:

$$L = - [w1 \cdot y \cdot \log(p) + (1 - y) \cdot \log(1 - p)]$$

where w1 is the weight given to the suspicious class. A larger w1 makes errors on suspicious accounts more costly, which pushes the model to pay more attention to them. The training procedure repeatedly performs a forward pass through the two layers, computes the loss, and updates the weights using a gradient-based optimiser for the configured number of epochs and the configured learning rate.

C. Risk Scoring

The model produces a probability-like risk score between 0 and 1. The dashboard maps

the score into LOW, MEDIUM and HIGH risk categories. A higher score indicates a higher priority for investigation.

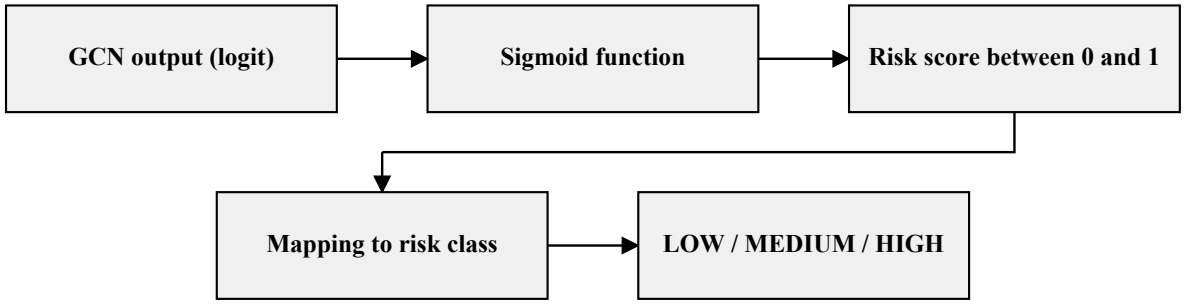


Fig 4.3 From Model Output to Risk Class

Table 4.3 Risk Categories

Category	Meaning for the investigator
LOW	Lowest priority: the model assigns a low risk score to the account.
MEDIUM	Intermediate priority: the account may deserve review when capacity allows.
HIGH	Highest priority: the account should be reviewed first, together with its transaction evidence.

D. Dashboard Processing

The Streamlit application loads saved model artifacts rather than retraining the model whenever the user navigates between dashboard pages. Caching is used to reduce repeated dataset preprocessing. Network visualization limits the displayed neighbourhood to avoid unnecessarily large interactive graphs.

E. Training Procedure

Training is started from the Model Training & Evaluation page. The steps are summarized in the following outline, which is written as pseudocode for clarity.

```
load transactions; validate fields; encode types
create account IDs; map accounts to node indices
compute account features; scale features; create labels
build edges: sender index -> receiver index
initialise two-layer GCN (64, 32), dropout 0.3
for epoch in 1..E:
    forward pass -> logits for all accounts
    loss = weighted BCEWithLogits(logits, labels)
    update weights (learning rate set by user)
evaluate: accuracy, precision, recall, F1, ROC-AUC
save gcn_model.pt, graph_data.pt, metrics.json, scaler.pkl
```

F. Network Visualization

To display the network of an account, the dashboard selects the account, collects the

transactions in which it takes part and the accounts at the other end of those transactions, and draws the resulting local neighbourhood. Limiting the number of displayed accounts keeps the visualization readable and responsive. The neighbourhood view reflects the same information that the GCN uses when it aggregates the neighbours of the account.

G. Report and Export

The Report / Export page collects the results of the analysis. Suspicious accounts and predictions can be exported to CSV files, and an investigation summary can be prepared. Because the exports contain financial information, they should be handled according to the security controls in Section 3.5.

4.2.2 RESULTS AND DISCUSSION

The evaluation results of the trained model are given in Table 4.4 and are plotted in Fig 4.4.

Table 4.4 Performance Evaluation Metrics

Metric	Result
Accuracy	99.94%
Precision	25.00%
Recall	3.03%
F1 Score	5.41%
ROC-AUC	71.95%

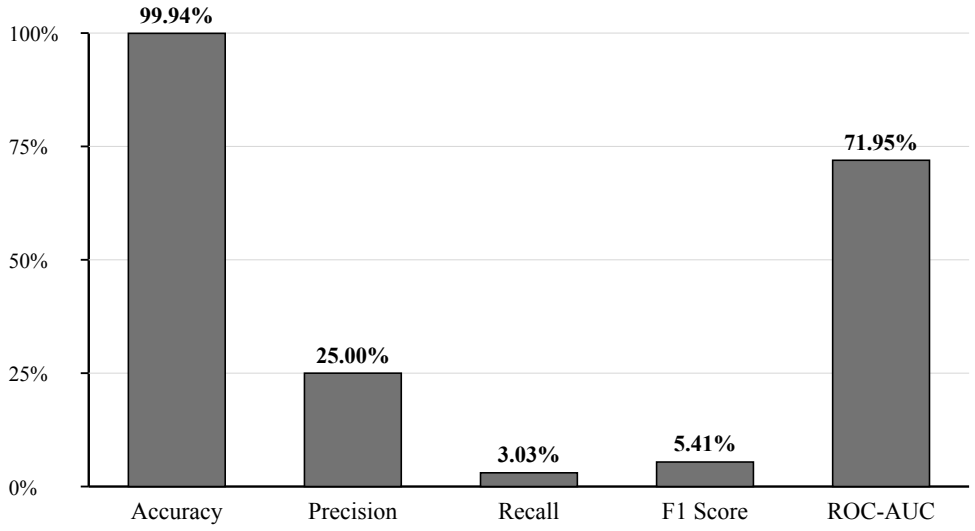


Fig 4.4 Evaluation Metrics of the Trained Model

Interpretation: The 99.94% accuracy should not be interpreted as 99.94% fraud-detection effectiveness because the dataset is highly imbalanced. Precision, recall,

F1-score and ROC-AUC are more meaningful for evaluating suspicious-activity detection.

To illustrate why accuracy is misleading, note that the 147 fraudulent records form only about 0.07% of the 200,000 transactions. A trivial classifier that labelled every transaction as legitimate would therefore be correct for roughly 99.93% of the records while detecting no fraud at all.

The recall of 3.03% indicates that the current operating threshold captures only a small proportion of positive cases. The ROC-AUC of 71.95% indicates useful ranking capability, but further model and threshold optimization is required.

Table 4.5 Interpretation of Evaluation Metrics

Metric	Meaning	Observation for this project
Accuracy	Share of all predictions that are correct	Very high, but dominated by the legitimate majority class
Precision	Share of flagged accounts that are truly suspicious	25.00%: some flagged accounts are suspicious
Recall	Share of suspicious accounts that are found	3.03%: few suspicious accounts are captured
F1 Score	Balance between precision and recall	5.41%: limited overall at this threshold
ROC-AUC	Ability to rank suspicious above normal accounts	71.95%: useful ranking ability

These measures are defined as follows, where TP, FP, TN and FN denote true positives, false positives, true negatives and false negatives (Fig 4.5):

		Actual class	
		Suspicious	**Normal**
Predicted suspicious		**TP** True Positive	**FP** False Positive
Predicted normal		**FN** False Negative	**TN** True Negative

Fig 4.5 Confusion Matrix Terms

$$Precision = TP / (TP + FP) \qquad Recall = TP / (TP + FN)$$

$$F1 = 2 \times Precision \times Recall / (Precision + Recall)$$

Table 4.6 Confusion Matrix Terms

Term	Meaning
True Positive (TP)	A suspicious account correctly flagged as suspicious.
False Positive (FP)	A normal account wrongly flagged as suspicious; costs investigator time.
False Negative (FN)	A suspicious account that was missed; the most serious error in AML.
True Negative (TN)	A normal account correctly left unflagged.

4.2.3 Discussion of the Results

Effect of class imbalance. With only 147 fraudulent records among 200,000 transactions, the model sees very few positive examples. Even with class weighting, learning a reliable decision boundary from so few examples is difficult. This is the main reason for the large gap between the accuracy and the other metrics.

Low recall. A recall of 3.03% means that most suspicious accounts are not flagged at the present threshold. In AML a missed suspicious account (a false negative) is generally more serious than a false alarm, so the low recall is the most important weakness of the current model. Lowering the decision threshold would flag more accounts and increase recall, at the cost of lower precision; selecting the threshold should therefore be treated as an explicit optimization step.

Precision and F1-score. A precision of 25.00% means that one in four flagged accounts was suspicious. Because the recall is low, the F1-score, which balances precision and recall, is also low at 5.41%.

Ranking ability. The ROC-AUC of 71.95% is clearly above the value of 50% expected from random ranking. This indicates that the risk scores contain useful information about which accounts are more likely to be suspicious, even though the default threshold does not convert that information into good classification. For an investigator who works down a ranked list, ranking quality is directly useful.

Implication. The results show that the full pipeline, from the transaction data to a ranked list of accounts with supporting evidence, works end to end. They also show that model and threshold optimization, the use of PR-AUC, richer features and more realistic data are needed before the detection performance can be considered satisfactory. These conclusions are reflected in the future enhancements in Chapter 5.

4.2.4 Training Settings and Their Effects

The dashboard lets the user choose the number of training epochs and the learning rate. These settings influence the result in well-understood ways. A very small number of epochs may leave the model under-trained, while a very large number may cause over-fitting to the training accounts. A learning rate that is too high can make the training unstable, and one that is too low makes it slow. Dropout and the sizes of the hidden layers (64 and 32) have a similar balancing role: larger layers can represent more complex patterns, and dropout limits the risk of memorizing the training data.

Because the suspicious class is extremely rare, the choice of settings should always be judged with precision, recall, F1-score and ROC-AUC and not with accuracy alone. The metrics are saved with the model in metrics.json so that different runs can be compared.

4.2.5 Decision Threshold and Ranking

The GCN outputs a risk score for each account, and a decision threshold converts this score into a flagged or not-flagged decision. Table 4.7 summarizes the qualitative effect of moving the threshold. The values shown are directions of change only; they are not measured results.

Table 4.7 Qualitative Effect of the Decision Threshold

Threshold change	Effect on recall	Effect on precision
Lower threshold	Tends to increase	Tends to decrease
Higher threshold	Tends to decrease	Tends to increase

The recall of 3.03% suggests that the current threshold is conservative. A lower threshold would flag more accounts, which would normally raise recall and reduce precision. Choosing the threshold requires a judgement about the cost of missing a suspicious account compared with the cost of investigating an innocent one. In practice, many institutions prefer to accept more false alarms in exchange for fewer missed cases, provided investigators have the capacity to review them.

Alternatively, the scores can be used purely as a ranking. An investigator who reviews accounts from the highest score downwards benefits from the ROC-AUC of 71.95%, because it shows that suspicious accounts tend to receive higher scores than normal ones. This ranking-based use is how the Risk Analysis module presents the top-risk accounts.

4.2.6 Comparison with Transaction-by-Transaction Analysis

Table 4.8 Qualitative Comparison of the Two Views

Aspect	Transaction-by-transaction view	Graph-based view (this project)
Information used	Fields of a single transaction	Account behaviour and neighbouring accounts
Multi-hop patterns	Not visible	Reflected through two GCN layers
Evidence for investigator	Single records	Transactions and local network
Main limitation	Misses distributed behaviour	Low recall on imbalanced data

4.2.7 Illustrative Investigation Scenario

The following scenario is hypothetical and is included only to show how the modules are used together; it does not describe actual results. An investigator opens the Risk Analysis page and sees that a number of accounts are in the HIGH category. She selects the account with the highest score and opens the Transaction Analysis page. The page shows the transaction count, the amounts involved, the connected accounts and the most recent transactions. She then opens the Network Visualization page and sees the local neighbourhood of the account. If the account exchanges funds with a small group of other accounts in a repeated pattern, she may mark it for further review and export it, together with the predictions, from the Report / Export page.

4.2.8 Limitations of the Evaluation

Several limitations should be kept in mind when reading the results. The evaluation uses a single synthetic dataset, so the metrics describe the behaviour of the model on that data and not on real transactions. The suspicious class contains very few examples, which makes every metric for that class sensitive to small changes. The decision threshold was not tuned, so the precision, recall and F1-score describe one operating point only. Finally, the system classifies accounts, whereas investigators may also be interested in groups of accounts or in sequences of transactions over time; these are possibilities for future work.

4.3 IMPLEMENTATION DETAILS

The project is organized as a small set of Python modules, listed in Table 4.9. Each module has one main responsibility, which makes the code easier to understand, test and extend.

Table 4.9 Project Files and Responsibilities

File	Responsibility
app.py	Streamlit dashboard: pages, charts, network view and export.

File	Responsibility
preprocessing.py	Loading, validation, encoding, feature calculation, label creation and scaling.
graph_builder.py	Mapping accounts to node indices and constructing the graph data.
model.py	Definition of the two-layer GCN.
train.py	Training loop, evaluation and saving of model artifacts.
predict.py	Loading saved artifacts and producing risk scores.

4.3.1 Saved Artifacts

Training saves four artifacts in the models directory: the trained model parameters (gcn_model.pt), the saved graph data (graph_data.pt), the evaluation metrics and training information (metrics.json) and the feature-scaling object (scaler.pkl). Saving the artifacts means that the model need not be retrained each time the dashboard is used, and that predictions at inference time use exactly the same scaling as training.

4.3.2 Performance Considerations

Three measures keep the dashboard responsive. First, saved artifacts are loaded rather than recomputed whenever the user changes page. Second, dataset preprocessing is cached so that it is not repeated. Third, the network visualization is restricted to a local neighbourhood so that the interactive graph remains small. Together these measures address the problem of long processing time identified in Section 1.2.

4.4 TESTING AND VERIFICATION

Verification of the system is organized by module. Table 4.10 lists the main checks that were designed for the dashboard and the behaviour expected in each case. The table describes the expected behaviour; it is not a record of measured results.

Table 4.10 Verification Checks by Module

No.	Module	Expected behaviour
1	Dataset	Dataset information, transaction types and fraud distribution are displayed.
2	Preprocessing	Invalid or malformed fields are rejected before graph construction.
3	Graph construction	Every account is mapped to a node and every transaction to a directed edge.
4	Training	Model, graph data, metrics and scaler are saved after training.

No.	Module	Expected behaviour
5	Evaluation	Accuracy, precision, recall, F1-score and ROC-AUC are reported.
6	Transaction analysis	A searched account shows its score, transactions and connected accounts.
7	Network visualization	A limited local neighbourhood is drawn for the selected account.
8	Risk analysis	Accounts are grouped into LOW, MEDIUM and HIGH categories.
9	Export	Suspicious accounts and predictions can be exported as CSV files.

4.4.1 Validation Approach

Two kinds of validation are relevant. Functional validation confirms that every dashboard module behaves as described in Table 4.10. Model validation evaluates the trained GCN with accuracy, precision, recall, F1-score and ROC-AUC, as reported in Section 4.2.2. Because the dataset is synthetic and highly imbalanced, the model validation results should be regarded as a demonstration of the pipeline and not as a measure of performance in a real institution.

CHAPTER 5
CONCLUSION

The project demonstrates a working Graph-Based Anti-Money Laundering Detection System using a Graph Convolutional Network.

The system converts financial transactions into a graph where Accounts = Nodes and Transactions = Edges. The GCN learns relationships between connected accounts and produces account-level risk scores.

The Streamlit dashboard integrates:

- Dataset analysis
- Model training
- Model evaluation
- Transaction analysis
- Network visualization
- Risk analysis
- Report generation

The project demonstrates how graph-based learning can provide relational information that traditional transaction-by-transaction analysis may miss.

The evaluation results give a balanced picture. The pipeline is complete and the dashboard allows an investigator to move from a ranked list of accounts to the supporting transactions and network. At the same time, the recall of 3.03% shows that the model, in its present form and on this highly imbalanced synthetic dataset, captures only a small proportion of suspicious accounts. The report therefore presents the system as a foundation for further work and not as a finished detection tool.

5.1 FUTURE ENHANCEMENTS

- Optimize classification threshold.
- Use PR-AUC for imbalanced datasets.
- Add temporal transaction features.
- Implement GraphSAGE.
- Implement Graph Attention Networks.
- Add Explainable AI.
- Add real-time transaction processing.
- Add authentication and role-based access.

- Add secure audit logging.
- Evaluate using realistic privacy-preserving datasets.

Table 5.1 Future Enhancements and Expected Benefit

Enhancement	Expected benefit
Threshold optimization	Higher recall by choosing a better operating point.
PR-AUC	A metric focused on the rare suspicious class.
Temporal features	Capture how account behaviour changes over time.
GraphSAGE	Neighbour sampling for larger graphs.
Graph Attention Networks	Weight neighbouring accounts differently.
Explainable AI	Show why an account received a high score.
Real-time processing	Score transactions as they arrive.
Authentication and role-based access	Protect investigation results.
Secure audit logging	Record who viewed or exported results.
Realistic privacy-preserving datasets	More credible evaluation.

Threshold optimization and PR-AUC address the low recall observed on the imbalanced dataset. Temporal features, GraphSAGE and Graph Attention Networks can capture how behaviour changes over time and weight neighbouring accounts differently. Explainable AI would help investigators understand why an account received a high score. Authentication, role-based access and audit logging are required before any real-world deployment.

5.2 LIMITATIONS

- The evaluation uses a single synthetic dataset.
- Recall at the current operating threshold is low.
- The model performs account-level classification only.
- The system is intended for academic demonstration, not production use.

5.3 SUMMARY OF CONTRIBUTIONS

- A transaction graph built from the PaySim dataset, with accounts as nodes and transactions as directed edges.
- A two-layer GCN that produces account-level risk scores and LOW, MEDIUM and HIGH risk categories.
- An evaluation that reports precision, recall, F1-score and ROC-AUC alongside

accuracy, with an explicit discussion of class imbalance.

- An interactive Streamlit dashboard that links model output to transaction evidence, network visualization, risk analysis and export.
- A threat model and a set of security controls suited to a financial-crime investigation tool.

Taken together, these contributions provide a foundation on which the enhancements listed in Section 5.1 can be built.

5.4 SOCIETAL AND ETHICAL CONSIDERATIONS

Automated systems that flag accounts as suspicious affect real people. A false positive can lead to an innocent customer being investigated, delayed or inconvenienced, while a false negative allows criminal activity to continue. For these reasons the proposed system presents risk scores only as investigation indicators, shows the supporting evidence and relies on human review. Any real deployment would also have to examine whether the model treats different groups of customers fairly and would have to comply with data-protection law.

5.5 RECOMMENDATIONS FOR FURTHER WORK

Based on the results, the most valuable next steps are, in order: to optimize the decision threshold and report PR-AUC so that the recall and the precision of the model on the rare class can be assessed properly; to add temporal features and test GraphSAGE and Graph Attention Networks, which may represent the data better than the present two-layer GCN; to evaluate the system on more realistic privacy-preserving data; and to add authentication, role-based access and audit logging so that the dashboard can be used safely with sensitive data.

REFERENCES

1. M. Weber, G. Domeniconi, J. Chen, D. K. I. Weidele, C. Bellei, T. Robinson and C. E. Leiserson, “Anti-Money Laundering in Bitcoin: Experimenting with Graph Convolutional Networks for Financial Forensics,” arXiv:1908.02591, 2019.
2. “GCN + Focal Loss for Class-Imbalanced AML Detection,” IEEE Xplore (GCN for AML Detection).
3. “Financial Fraud Detection Using Graph Neural Networks: A Systematic Review,” ScienceDirect (GNN Financial Fraud Review).
4. “Graph Contrastive Pre-training for Anti-Money Laundering,” Springer, DOI: 10.1007/s44196-024-00720-4.
5. C. Bellei, M. Xu, R. Phillips, T. Robinson, M. Weber, T. Kaler, C. E. Leiserson, Arvind and J. Chen, “The Shape of Money Laundering: Subgraph Representation Learning on the Blockchain with the Elliptic2 Dataset,” arXiv:2404.19109, 2024.
6. “Global-Local Graph Attention for Bitcoin AML Detection,” Scientific Reports.
7. “Elliptic GNN Implementation / Reference Project,” GitHub (AML Elliptic GNN).
8. “Heterogeneous Graph Modeling for AML,” GitHub (Heterogeneous Graph AML Study).

APPENDIX

A. PROJECT DIRECTORY STRUCTURE

```
Graph_AML_Project/
├── app.py
├── preprocessing.py
├── graph_builder.py
├── model.py
├── train.py
├── predict.py
├── data/
├── models/
│   ├── gcn_model.pt
│   ├── graph_data.pt
│   ├── metrics.json
│   └── scaler.pkl
└── outputs/
```

B. MODEL ARTIFACTS

- **gcn_model.pt** – trained GCN model parameters.
- **graph_data.pt** – saved graph data.
- **metrics.json** – evaluation metrics and training information.
- **scaler.pkl** – feature-scaling object used during inference.

C. GLOSSARY OF TERMS

Table A.1 Glossary of Terms

Term	Description
AML	Anti-Money Laundering: measures to detect and prevent the concealment of illicit funds.
Node	A graph element; here, a financial account.
Edge	A directed link between nodes; here, a transaction from one account to another.
GCN	Graph Convolutional Network: a neural network that aggregates information from neighbouring nodes.
Risk score	A value between 0 and 1 produced by the model for each account.
PaySim	The synthetic financial transaction dataset used in this project.
Precision	Share of flagged accounts that are truly suspicious.
Recall	Share of suspicious accounts that are correctly identified.
ROC-AUC	Measure of how well the model ranks suspicious accounts above normal ones.
Streamlit	The framework used to build the interactive dashboard.

Term	Description
Class imbalance	A situation in which one class is far rarer than the other.
Message passing	Exchange of information between neighbouring nodes in a GNN layer.

D. USING THE DASHBOARD

A typical investigation session follows the steps below, using the modules described in Section 3.2.

1. Open the Home page to review the project overview, dataset status, model status and key metrics.
2. Open the Dataset page to inspect transaction records, transaction types and the fraud distribution.
3. Use Model Training & Evaluation to set the training epochs and learning rate, train the model and review the evaluation metrics.
4. Use Transaction Analysis to search for an account and review its risk score, transaction count, connected accounts, financial summary and recent transactions.
5. Use Network Visualization to view the account, its transaction edges and its local neighbourhood.
6. Use Risk Analysis to view LOW, MEDIUM and HIGH risk accounts, the risk distribution and the top-risk accounts.
7. Use Report / Export to export suspicious accounts and predictions and to prepare the investigation summary.

E. TECHNOLOGIES USED

Table A.2 Technologies Used

Technology	Role in the project
Python	Implementation language for the pipeline and dashboard.
PyTorch / PyTorch Geometric	Implementation, training and evaluation of the Graph Convolutional Network.
NetworkX	Graph representation and network analysis of accounts and transactions.
Pandas	Loading, validation and preprocessing of transaction data.
Streamlit	Interactive dashboard for analysis, visualization and export.

F. PSEUDOCODE OF THE MAIN STEPS

The following outlines describe the main processing steps in simplified pseudocode. They are written for explanation and do not reproduce the source code.

```
PREPROCESS(transactions):
    validate required fields; reject malformed records
    encode transaction types as numbers
    assign an ID to every distinct account
    map account IDs to node indices 0..N-1
    features[node] = summary of the account behaviour
    labels[node]       = suspicious if involved in fraudulent records
    features = scale(features)                    # scaler is saved
    edges = [(sender_index, receiver_index) for each transaction]
    return features, labels, edges

SCORE_AND_CLASSIFY(model, graph):
    logits = model(graph.features, graph.edges)
    score  = sigmoid(logits)                      # value between 0 and 1
    risk_class = LOW / MEDIUM / HIGH according to the score
    return score, risk_class
```

G. QUESTIONS AND ANSWERS ABOUT THE PROJECT

Q. Why use a graph instead of a table of transactions?

A. Because suspicious behaviour is often spread over several connected accounts. A graph lets the model combine an account with its neighbours, which a table of independent rows cannot do directly.

Q. Why is the accuracy so high while the recall is low?

A. The dataset is highly imbalanced, so a model that rarely flags any account is still correct for almost all records. Accuracy is therefore dominated by the legitimate majority class, and precision, recall, F1-score and ROC-AUC are the meaningful measures.

Q. What does the ROC-AUC of 71.95% tell us?

A. It shows that the model ranks suspicious accounts above normal accounts more often than chance would. The risk scores are therefore useful for prioritizing accounts, although the present threshold is not yet effective for classification.

Q. Why does the GCN have two layers?

A. Two layers let information from accounts up to two transactions away influence an account, which captures multi-hop behaviour while keeping the model simple.

Q. How is class imbalance handled?

A. By class weighting in the BCEWithLogitsLoss, which gives the rare suspicious class a larger contribution to the loss.

Q. Why is synthetic data used?

A. Real financial data is confidential. Synthetic data allows the system to be developed and

demonstrated without exposing customer information, although results may not carry over to real institutions.

Q. How is the model protected?

A. Model files are stored in a separate directory with restricted access, and a real deployment would add authentication, encryption and audit logging.

H. LIST OF ABBREVIATIONS

Table A.3 List of Abbreviations

Abbreviation	Expansion
AML	Anti-Money Laundering
AUC	Area Under the Curve
BCE	Binary Cross-Entropy
CSV	Comma-Separated Values
GAT	Graph Attention Network
GCN	Graph Convolutional Network
GNN	Graph Neural Network
PR-AUC	Area under the Precision-Recall curve
ROC	Receiver Operating Characteristic
SDG	Sustainable Development Goal
FN / FP	False Negative / False Positive
TN / TP	True Negative / True Positive

I. CHECKLIST FOR SECURE DEPLOYMENT

The following checklist summarizes the measures recommended in Section 3.5 for any deployment beyond academic demonstration.

Table A.4 Deployment Checklist

No.	Measure	Related threat
1	Authenticate users and apply role-based authorization.	Dashboard abuse
2	Validate every input file before processing.	Malicious CSV
3	Restrict write access to model files to the training component.	Model tampering
4	Encrypt stored data and exported reports.	Data leakage
5	Keep an audit log of views and exports.	Data leakage, abuse
6	Apply retention and deletion rules to financial data.	Data leakage

No.	Measure	Related threat
7	Monitor model performance and retrain when behaviour changes.	Adversarial transactions
8	Use risk scores only as indicators for human review.	Over-reliance on scores

J. METRIC DEFINITIONS

Let TP, FP, TN and FN be the numbers of true positives, false positives, true negatives and false negatives for the suspicious class.

Table A.5 Definitions of the Evaluation Metrics

Metric	Definition	Remark
Accuracy	(TP + TN) / (TP + TN + FP + FN)	Dominated by the majority class
Precision	TP / (TP + FP)	Quality of the flags
Recall	TP / (TP + FN)	Coverage of suspicious accounts
F1 Score	2PR / (P + R)	Balance of precision and recall
ROC-AUC	Area under the TPR versus FPR curve	Threshold-independent ranking quality

In the table, P and R denote precision and recall, TPR is the true positive rate (equal to recall) and FPR is the false positive rate, FP / (FP + TN). The ROC curve is obtained by varying the decision threshold and plotting the pair (FPR, TPR) for every threshold; the area under this curve is the ROC-AUC. A value of 50% corresponds to random ranking and 100% to perfect ranking.

K. DASHBOARD PAGE ELEMENTS

Table A.6 lists, for each dashboard module, the controls offered to the user and the information displayed, as described in Section 3.2.

Table A.6 Dashboard Page Elements

Module	User controls	Information displayed
Home	Navigation menu	Project overview, dataset status, model status, key metrics, architecture
Dataset	Page navigation	Dataset information, transaction records, transaction types, fraud distribution, statistics, samples
Model Training and Evaluation	Dataset configuration, epochs, learning rate, train button	Evaluation metrics: accuracy, precision, recall, F1-score, ROC-AUC
Transaction Analysis	Account search	Risk score, transaction count, connected accounts, financial summary, recent transactions
Network Visualization	Account selection	Account nodes, transaction edges, connected accounts, local neighbourhood
Risk Analysis	Page navigation	LOW, MEDIUM and HIGH accounts, risk distribution, top-risk accounts
Report / Export	Export options	Suspicious-account export, prediction export, investigation summary

L. PREPROCESSING STEPS, PURPOSE AND OUTPUT

Table A.7 Summary of Preprocessing Steps

Step	Purpose	Output
Load transaction data	Bring the records into memory	Transaction table
Validate fields	Reject malformed input	Clean transaction table
Encode transaction types	Convert categories to numbers	Numeric type codes
Create account IDs	Identify every account uniquely	Account identifiers
Map accounts to node indices	Number the graph nodes	Node index per account
Calculate account features	Summarize account behaviour	Feature matrix
Construct edges	Represent transfers of funds	Directed edge list
Create account labels	Provide the supervised target	Label per account
Scale numerical features	Balance feature ranges	Scaled features, scaler.pkl

M. REQUIREMENTS AND VERIFICATION

Table A.8 relates the functional requirements of Table 3.3 to the verification checks of Table 4.10.

Table A.8 Requirements and Verification Checks

Requirement	Verification check
FR1	Check 1 (Dataset) and check 2 (Preprocessing)
FR2	Check 2 (Preprocessing)
FR3	Check 3 (Graph construction)
FR4	Checks 4 and 5 (Training and Evaluation)
FR5	Check 6 (Transaction analysis)
FR6	Check 8 (Risk analysis)
FR7	Check 6 (Transaction analysis)
FR8	Check 7 (Network visualization)
FR9	Check 9 (Export)

N. TRANSACTION PATTERNS RELEVANT TO GRAPH ANALYSIS

The structural patterns in Table A.9 are commonly discussed in the analysis of money-laundering networks. They are given here to explain why neighbourhood information can be informative; the system does not search for these patterns explicitly, and the GCN learns from the data which structures are associated with suspicious labels.

Table A.9 Structural Patterns in Transaction Graphs

Pattern	Description	Why relationships matter
Chain	Funds pass through a sequence of accounts	Each hop looks ordinary; the sequence does not
Fan-out	One account sends funds to many accounts	Splitting is visible only in the neighbourhood
Fan-in	Many accounts send funds to one account	Collection is visible only in the neighbourhood
Cycle	Funds return to the account that started them	Needs the path structure, not single records
Dense cluster	A small group of accounts transact heavily with each other	Group behaviour differs from individual behaviour

O. GRAPH NEURAL NETWORK ARCHITECTURES

The present project uses a GCN. Table A.10 compares it with the architectures proposed as future enhancements.

Table A.10 Comparison of Graph Neural Network Architectures

Architecture	How neighbours are combined	Relevance to this project
GCN	Normalized averaging of a node and its neighbours	Implemented in the present system
GraphSAGE	Aggregation over sampled neighbours	Future enhancement; scalability to larger graphs
Graph Attention Network	Learned weights for each neighbour	Future enhancement; neighbours weighted differently

GraphSAGE learns how to aggregate information from a sample of neighbours, which allows it to scale to large graphs and to produce representations for accounts that were not present during training. Graph Attention Networks learn an importance weight for every neighbour, so that a model can pay more attention to the accounts that matter most for a prediction. Both are natural next steps once the baseline GCN has been fully evaluated.

P. NOTES ON REPRODUCING THE RESULTS

The reported metrics can be reproduced by following the sequence below with the PaySim data used in the project.

1. Place the PaySim transaction file in the data directory.
2. Open the dashboard and use the Dataset page to confirm that 200,000 transactions have been processed and that 147 are fraudulent.
3. Open Model Training & Evaluation, set the training epochs and learning rate, and train the model.
4. Read the evaluation metrics from the page; they are also stored in metrics.json in the models directory.
5. Open Risk Analysis and Transaction Analysis to examine the resulting risk scores and the supporting evidence.

Because training involves random initialization and random dropout, small differences between runs are normal unless the random seed is fixed. The metrics reported in this document are those obtained for the trained model that accompanies the project.

Q. SUMMARY OF KEY FIGURES

Table A.11 Key Figures of the Project

Item	Value
Dataset	PaySim (synthetic)
Transactions processed	200,000
Fraudulent transactions	147
Approximate fraud rate	0.07%
GCN layers	2 (64 and 32 hidden dimensions)
Dropout	0.3
Loss function	BCEWithLogitsLoss with class weighting
Accuracy	99.94%
Precision	25.00%
Recall	3.03%
F1 Score	5.41%
ROC-AUC	71.95%

R. MAPPING OF OBJECTIVES TO REPORT SECTIONS

Table A.12 shows where each objective of the project is described and evaluated in this report.

Table A.12 Objectives and Report Sections

Objective	Described in
Construct a graph from transaction data	Sections 4.2 and 4.2.1 (A)
Represent accounts as nodes and transactions as directed edges	Section 4.2.1 (A), Fig 4.1
Generate account-level behavioural features	Section 4.2
Implement a Graph Convolutional Network	Section 4.2.1 (B), Fig 4.2
Identify suspicious accounts and generate risk scores	Sections 4.2.1 (C) and 4.2.2
Classify accounts into LOW, MEDIUM and HIGH risk	Section 4.2.1 (C), Table 4.3
Provide transaction-level investigation	Section 3.2.4
Visualize account relationships	Sections 3.2.5 and 4.2.1 (F)
Provide report and export functionality	Sections 3.2.7 and 4.2.1 (G)

Each objective stated in Section 1.3 is therefore addressed by a design description in Chapter 3 and by an implementation or evaluation description in Chapter 4.